Департамент образования и науки Брянской области
Государственное автономное профессиональное образовательное учреждение
«Новозыбковский профессионально-педагогический колледж»

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА К КУРСОВОМУ ПРОЕКТУ

Разработка веб-приложения для конверсии медиафайлов

Яриго Иван Николаевич
Специальность: 09.02.07
Информационные системы и программирование
III курс, 35 группа
Научный руководитель:
Саросек Сергей Михайлович
преподаватель информатики, спецдисциплин

Новозыбков, 2026

Содержание

Аннотация 4

Введение 6

Глава 1. Теоретические основы асинхронной обработки данных 8

1.1. Понятие асинхронности в веб-приложениях 8

1.1.1. Определение и основные концепции 8

1.1.2. Модель асинхронной обработки на основе очередей задач ... 9

1.1.3. Преимущества и недостатки асинхронного подхода 11

1.1.3.1. Преимущества асинхронной обработки 11

1.1.3.2. Недостатки и ограничения асинхронного подхода 13

1.1.3.3. Итоги анализа 14

1.1.4. Типичные сценарии использования 14

1.2. Очереди задач: брокеры сообщений (RabbitMQ, Apache Kafka) и реляционные базы данных 16

1.2.1. Реляционные базы данных как очередь 16

1.2.2. Специализированные брокеры сообщений 17

1.2.2.1. RabbitMQ 17

1.2.2.2. Apache Kafka 18

1.2.3. Сравнительный анализ существующих решений 19

1.2.4. Итоги анализа 22

1.3. Принципы работы S3-совместимых объектных хранилищ (MinIO, Garage, SeaweedFS, Ceph) 23

1.4. Сравнительный анализ существующих решений и обоснование выбора технологий 23

Глава 2. Проектирование архитектуры веб-приложения 24

2.1. Схема взаимодействия компонентов сервиса 24

2.2. Проектирование базы данных 24

2.3. Определение функциональных требований и REST API..... 24

2.4. Организация асинхронной обработки задач: двухуровневая очередь и планировщик Spring 24

2.5. Пул воркеров с ограниченным числом потоков..... 24

Глава 3. Реализация ключевых компонентов..... 25

3.1. Серверная часть приложения: Java 25, Spring Boot 4, Spring Data JPA..... 25

3.2. Асинхронная обработка: ThreadPoolTaskExecutor, планировщик задач..... 25

3.3. Интеграция с S3-хранилищем через AWS SDK v2..... 25

3.4. Конвертация медиафайлов с использованием FFMpeg через JavaCV 25

3.5. Клиентская часть приложения: TypeScript, React, паттерн «конечный автомат», взаимодействие с API..... 25

3.6. Контейнеризация и развёртывание: Docker, docker-compose, reverse proxy Caddy, HTTPS 25

3.7. Тестирование основных сценариев работы и оценка производительности..... 25

Заключение 26

Список использованной литературы..... 27

Приложение 29

Аннотация

Данный курсовой проект посвящён разработке асинхронного веб-сервиса для конвертации медиафайлов (изображений, аудио, видео). Целью работы является создание отказоустойчивой системы, позволяющей пользователям загружать файлы, получать идентификатор задачи немедленно и скачивать результат после её обработки.

Пояснительная записка состоит из введения, трёх глав, заключения, списка использованной литературы и двух приложений. Общий объём записки составляет ?? страницы.

В первой главе рассматриваются теоретические основы асинхронной обработки данных: понятие асинхронности, способы организации очередей задач (на основе RabbitMQ, Kafka и реляционных баз данных), а также принципы работы объектных хранилищ, совместимых с S3 (MinIO, Garage, Ceph, SeaweedFS). Проведён сравнительный анализ существующих решений и обоснован выбор технологий для реализации проекта.

Во второй главе описано проектирование архитектуры сервиса. Разработана схема взаимодействия компонентов, спроектирована база данных (таблица conversion_jobs для хранения задач), определены функциональные требования и REST API (эндпоинты для создания задачи, получения статуса, скачивания файла и группового получения статусов задач). Особое внимание уделено организации асинхронной обработки: предложена двухуровневая очередь на основе PostgreSQL и встроенного планировщика Spring, а также пул воркеров с ограниченным числом потоков.

В третьей главе представлена реализация ключевых компонентов. Серверная часть приложения разработана на Java 25 с использованием Spring Boot 4, Spring Data JPA; для многопоточной обработки задействован

ThreadPoolTaskExecutor. Интеграция с S3-хранилищем выполнена через библиотеку взаимодействия с S3-совместимыми хранилищами AWS SDK v2, для конвертации медиафайлов используется FFmpeg через библиотеку JavaCV. Клиентская часть приложения реализована на TypeScript с использованием React, с применением паттерна «конечный автомат» для управления состоянием, и периодического опроса статуса задач. Выполнена контейнеризация всех сервисов (Docker, Docker Compose), настроен reverse proxy (Caddy) с автоматическим HTTPS с TLS сертификатами от Let’s Encrypt. Проведено тестирование основных сценариев работы.

Практическая значимость проекта заключается в создании готового к использованию сервиса, который может быть внедрён для автоматизации конвертации медиафайлов. Разработанная архитектура проекта обеспечивает масштабируемость и может служить основой для более сложных систем обработки данных.

Ключевые слова: асинхронная обработка, очередь задач, Java, Spring Boot, TypeScript, React, S3-хранилище, Docker, многопоточность.

Введение

Современные веб-приложения всё чаще сталкиваются с необходимостью обработки больших объёмов медиаданных – изображений, аудио и видео. Пользователи ожидают быстрого отклика, однако конвертация медиафайлов является ресурсоёмкой и может занимать значительное время. Выполнение таких операций в синхронном режиме приводит к блокировке интерфейса и неэффективному использованию серверных ресурсов. В связи с этим актуальна разработка асинхронных систем, способных принимать задачи на обработку и выполнять их в фоновом режиме.

Объектом исследования является процесс асинхронной обработки медиафайлов в веб-приложениях.

Предмет исследования – методы и средства организации очереди задач, интеграции с объектным хранилищем (S3-совместимым) и реализации веб-интерфейса для управления конвертацией.

Цель работы заключается в разработке отказоустойчивого асинхронного веб-сервиса для конвертации медиафайлов, позволяющего пользователям загружать файлы, немедленно получать идентификатор задачи и скачивать результат после завершения фоновой обработки.

Задачи работы. Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести анализ существующих подходов к организации асинхронной обработки данных, включая использование очередей на основе реляционных баз данных и специализированных брокеров сообщений, а также изучить принципы работы S3-совместимых хранилищ.

2. Спроектировать архитектуру сервиса, включая схему взаимодействия компонентов, структуру базы данных для хранения заданий и REST API для управления задачами.
3. Разработать бэкенд на Java Spring Boot, реализующий пул воркеров, планировщик задач и интеграцию с S3-хранилищем (MinIO).
4. Создать клиентскую часть на TypeScript React с использованием конечного автомата для управления состоянием и реализовать взаимодействие с API.
5. Выполнить контейнеризацию всех компонентов с помощью Docker, настроить reverse proxy (Caddy) с автоматическим HTTPS и развернуть сервис на удалённом сервере.
6. Провести тестирование основных сценариев работы и оценить производительность системы.

Методы исследования включают анализ научно-технической литературы, сравнительный анализ, проектирование информационных систем, объектно-ориентированное программирование, интеграционное тестирование.

Практическая значимость работы заключается в создании готового к использованию сервиса, который может быть внедрён в систему обработки данных для автоматизации конвертации медиафайлов.

Глава 1. Теоретические основы асинхронной обработки данных

1.1. Понятие асинхронности в веб-приложениях

1.1.1. Определение и основные концепции

В классической модели взаимодействия компонентов веб-приложения преобладает синхронный подход. При синхронном взаимодействии клиент отправляет запрос серверу и переходит в состояние ожидания ответа. До тех пор, пока сервер не обработает запрос и не вернёт результат, клиент не может продолжить выполнение других операций. Такая модель хорошо знакома по стандартному HTTP-протоколу, где каждый запрос предполагает немедленный ответ [1].

Однако при выполнении длительных операций – отправке электронных писем, генерации отчётов, обработке загруженных изображений или взаимодействии с внешними API – синхронный подход обнаруживает существенные недостатки. Главный из них – неэффективное использование вычислительных ресурсов. Поток выполнения, выделенный для обработки запроса, простаивает в ожидании завершения операции, хотя мог бы быть занят другой полезной работой. В условиях высокой нагрузки это приводит к исчерпанию пула потоков и, как следствие, к отказу в обслуживании новых пользователей. Кроме того, пользователь вынужден наблюдать длительную загрузку страницы, что негативно сказывается на опыте взаимодействия с приложением.

Альтернативой синхронному подходу выступает асинхронное взаимодействие. В этом случае клиент отправляет запрос, но не дожидается его немедленного выполнения. Сервер немедленно подтверждает получение запроса и гарантирует, что задача будет обработана позже. Клиент может продолжать работу, а о результате обработки он узнает дополнительным способом – через повторный запрос, webhook или уведомление по

электронной почте. Такая модель аналогична отправке письма: отправитель не стоит у почтового ящика в ожидании ответа, а занимается своими делами [1; 4, с. 390].

На практике асинхронность часто реализуется с помощью очередей задач, которые будут детально рассмотрены в следующем пункте.

1.1.2. Модель асинхронной обработки на основе очередей задач

Как было показано в предыдущем разделе, синхронное выполнение длительных операций приводит к неэффективному использованию ресурсов и ухудшению пользовательского опыта. Решением этой проблемы является переход к асинхронной модели взаимодействия, при которой инициатор запроса не дожидается немедленного завершения операции, а продолжает свою работу, полагаясь на то, что задача будет выполнена позже.

Одним из распространённых способов организации асинхронной обработки в веб-приложениях является использование очередей задач. Эта модель предполагает наличие трёх основных компонентов: поставщика задач (producer или publisher), очереди (queue) и темы (topic), и потребителя (consumer или subscriber) [2; 5, с. 6-7]. В отличие от классической событийно-ориентированной архитектуры, где компоненты реагируют на произошедшие события предметной области [3], в модели очередей задач каждое сообщение интерпретируется как явная команда на выполнение конкретной работы (например, «отправить электронное письмо» или «сгенерировать отчёт») [2]. Такой подход проще в реализации и хорошо подходит для задач, требующих гарантированного выполнения и масштабирования обработчиков.

Рассмотрим каждый из элементов подробнее:

1. Поставщик задач. Это компонент приложения (например, веб-сервер или обработчик API-запроса), который создаёт задачу и помещает её в очередь. Поставщик не занимается непосредственным

выполнением задачи, а лишь формирует её описание (часто в виде JSON- или XML-сообщения) и передаёт в промежуточное хранилище. Благодаря этому время ответа пользователю сокращается до минимума [2; 5, с. 6].

2. Очередь. Представляет собой буфер, который надёжно хранит поступившие задачи до момента их обработки. Очередь может быть реализована как на основе специализированных брокеров сообщений (RabbitMQ, Apache Kafka), так и с использованием реляционных баз данных. Ключевые свойства очереди – сохранность задач и гарантия их доставки одному из потребителей [2].
3. Потребитель. Это отдельный сервис или процесс (часто называемый воркером), который извлекает задачи из очереди и выполняет предписанные действия. Потребители могут масштабироваться горизонтально: при увеличении нагрузки достаточно запустить дополнительные экземпляры воркеров, которые будут параллельно забирать задачи из очереди. Такой подход обеспечивает высокую пропускную способность системы [2; 5, с. 7].

В зависимости от требований приложения результат обработки может быть либо проигнорирован (модель «отправил и забыл»), либо записан в постоянное хранилище (базу данных или объектное хранилище) с последующим уведомлением пользователя. Например, после обработки загруженного изображения потребитель может сохранить его в S3-совместимом хранилище и обновить статус задачи в базе данных, что связывает данную модель с тематикой следующих разделов работы.

Несмотря на внешнее сходство, модель очередей задач следует отличать от полноценной событийно-ориентированной архитектуры (Event-Driven Architecture, EDA). В EDA основным объектом передачи являются события предметной области (например, «Заказ оплачен» или «Товар добавлен в

корзину»), на которые могут подписываться множество независимых обработчиков, запуская цепочки реакций [3]. В очередях задач сообщение, как правило, адресовано конкретному обработчику и подразумевает выполнение строго определённой работы [2]. Для задач асинхронной обработки данных в веб-приложениях модель очередей является более простой и часто достаточной.

1.1.3. Преимущества и недостатки асинхронного подхода

Переход от синхронной модели взаимодействия к асинхронной обработке данных в веб-приложениях сопряжён с рядом существенных преимуществ, однако требует также учёта определённых ограничений и архитектурных сложностей. В данном разделе проведён анализ сильных и слабых сторон асинхронного подхода, позволяющий обосновать его применимость для решения задач фоновой обработки данных.

1.1.3.1. Преимущества асинхронной обработки

1. Повышение отзывчивости пользовательского интерфейса. В синхронных приложениях выполнение длительной операции (например, отправка большого объёма данных на сервер или выполнение сложного запроса к базе данных) блокирует поток выполнения. Это приводит к «заморозке» интерфейса: пользователь не может взаимодействовать с приложением до завершения операции. Асинхронный подход устраняет эту проблему: задача инициируется, но выполняется в фоновом режиме, а поток управления немедленно возвращается к обработке событий интерфейса. Пользователь продолжает работу, а о результате операции он уведомляется позже. Такая модель критически важна

для современных одностраничных приложений (SPA) и мобильных интерфейсов, где плавность взаимодействия напрямую влияет на удовлетворённость пользователя [1; 4, с. 390; 6, с. 3].

2. Эффективное использование ресурсов и масштабируемость. При синхронной обработке каждый входящий запрос занимает поток (или процесс) на всё время своего выполнения, включая периоды ожидания ввода-вывода (I/O) – например, ответа от внешнего API, чтения с диска или выполнения запроса к базе данных. В это время поток простаивает, не принося пользы. Асинхронная модель позволяет освободить поток во время I/O-ожидания и задействовать его для обработки других запросов. Благодаря этому при том же объёме аппаратных ресурсов сервер способен обслуживать значительно большее количество параллельных операций. Особенно ярко это преимущество проявляется в сочетании с очередями задач. Поставщик задач лишь помещает сообщение в очередь и сразу освобождается, а обработчики работают в своём темпе. Если интенсивность поступления задач временно превышает пропускную способность обработчиков, задачи накапливаются в очереди, но не теряются и не приводят к отказу в обслуживании. При необходимости количество обработчиков можно увеличить горизонтально (например, запустить дополнительные экземпляры воркеров), что делает систему легко масштабируемой. [1; 4, с 390; 6, с. 3].
3. Повышение надёжности и отказоустойчивости. Использование очередей сообщений обеспечивает сохранность задач даже при сбоях компонентов. Большинство промышленных брокеров поддерживают персистентность: сообщения записываются на диск до того, как будут обработаны. Если обработчик падает во время выполнения задачи, сообщение не теряется – оно возвращается в очередь (или остаётся в ней) и может быть обработано другим

экземпляром обработчика после перезапуска. Это гарантирует доставку как минимум один раз (at-least-once) и существенно повышает общую надёжность системы. Кроме того, асинхронная архитектура позволяет изолировать сбои. Проблемы в одном из потребителей не влияют на работу поставщиков и на доступность основного приложения для пользователей. Отказ потребителя приводит лишь к временному накоплению задач в очереди, но не к потере данных [7, раздел «Reliability Guide»; 5, с. 161-180].

1.1.3.2. Недостатки и ограничения асинхронного подхода

1. Усложнение архитектуры и разработки. Внедрение асинхронной обработки неизбежно ведёт к появлению дополнительных компонентов: брокеров сообщений, очередей, воркеров, хранилищ результатов. Это усложняет инфраструктуру, требует её администрирования и мониторинга. Сам код с использованием колбэков (callback) или корутин (coroutine) может быть менее очевидным для разработчиков, привыкших к синхронному стилю, хотя современные языковые конструкции (async/await) частично смягчают эту проблему [6, с. 3].
2. Повышенная задержка для отдельных операций. Асинхронность не является серебряной пулей. Для очень коротких операций, которые выполняются за миллисекунды, накладные расходы на создание задачи, передачу через очередь и управление состоянием могут превысить выгоду от распараллеливания. Кроме того, асинхронная модель не подходит для задач, требующих немедленного ответа пользователю – например, для проверки подлинности или получения данных для отрисовки страницы. В таких сценариях синхронное взаимодействие остаётся более уместным. Также важно различать

I/O-bound и CPU-bound задачи: асинхронность эффективна только для первых, тогда для вычислительно интенсивных задач лучше использовать многопоточность или распределённые вычисления [8, с. 182, 204].

1.1.3.3. Итоги анализа

Проведённый анализ показывает, что асинхронный подход предоставляет значительные преимущества с точки зрения отзывчивости, эффективности использования ресурсов, масштабируемости и надёжности, особенно для задач, связанных с интенсивным вводом-выводом и фоновой обработкой. Однако его внедрение сопряжено с усложнением архитектуры, необходимостью тщательного проектирования и учёта потенциальных проблем согласованности данных. Для задач, подобных рассматриваемой в данной работе (асинхронная обработка пользовательского контента с сохранением в объектное хранилище), преимущества асинхронной модели существенно перевешивают её недостатки, что делает её обоснованным выбором для реализации практической части исследования.

1.1.4. Типичные сценарии использования

Асинхронная модель взаимодействия применяется в веб-приложениях повсеместно для решения задач, которые невозможно или нерационально выполнять синхронно в потоке пользовательского запроса. Наиболее распространённые сценарии включают:

1. Отправка электронной почты и уведомлений. Формирование и отправка писем через внешние SMTP-серверы или push-сервисы

может занимать несколько секунд. Асинхронная постановка в очередь позволяет немедленно вернуть пользователю ответ об успешном принятии запроса, гарантируя при этом доставку уведомления.

2. Обработка загружаемых медиафайлов. Конвертация видео или аудио в различные форматы являются ресурсоёмкими операциями. Выполнение таких задач в фоновых воркерах предотвращает блокировку веб-сервера и позволяет масштабировать обработку в зависимости от нагрузки.
3. Генерация отчётов и выгрузка документов. Создание сложных PDF-отчётов или Excel-таблиц по запросу пользователя часто требует времени. Асинхронный подход позволяет поставить задачу в очередь, а затем уведомить пользователя о готовности файла, например, по ссылке для скачивания.
4. Взаимодействие с внешними API. Если для ответа пользователю необходимо агрегировать данные из нескольких внешних источников, асинхронные HTTP-запросы позволяют выполнять их параллельно, сокращая общее время ожидания.

Все перечисленные сценарии объединяет общая черта: они либо являются длительными (секунды и минуты), либо связаны с ожиданием внешних сервисов (I/O-bound), либо критически важны с точки зрения гарантии доставки. Именно для таких задач асинхронная обработка на основе очередей является стандартом индустрии
[1; 4, с. 390; 8, с. 182, 204].

В рамках данной работы в качестве целевого сценария выбрана задача асинхронной обработки пользовательских медиафайлов с сохранением в объектное хранилище, которая обладает всеми перечисленными характеристиками. Это обуславливает выбор очереди задач и S3-

совместимого хранилища в качестве ключевых компонентов проектируемого решения.

1.2. Очереди задач: брокеры сообщений (RabbitMQ, Apache Kafka) и реляционные базы данных

1.2.1. Реляционные базы данных как очередь

Как было показано в разделе 1.1.2, центральным элементом асинхронной обработки является очередь, в которую поставщики помещают задачи и из которой потребители их извлекают. В простейшем случае функцию такой очереди может выполнять обычная реляционная база данных, уже используемая в приложении для хранения основной информации.

Идея заключается в создании специальной таблицы (например, jobs или tasks), где каждая запись соответствует отдельной задаче. Типичная структура такой таблицы включает идентификатор задачи, её тип, параметры (часто в гибком формате JSON), статус (например, pending, processing, done, failed), время создания и попыток выполнения. Потребители периодически опрашивают таблицу, выбирая задачи со статусом pending, и после успешной обработки обновляют статус на done. Для предотвращения повторного выбора одной задачи разными потребителями используются блокировки на уровне строк (например, `SELECT ... FOR UPDATE` в PostgreSQL или MySQL) [9].

Основным преимуществом использования реляционной БД в качестве очереди является простота и отсутствие дополнительных инфраструктурных компонентов. Если приложение уже использует базу данных, такой подход не требует развёртывания и администрирования отдельного брокера сообщений.

Кроме того, транзакционная модель ACID позволяет гарантировать, что изменение статуса задачи и связанные с ней изменения других данных будут выполнены атомарно — это важно для систем, требующих строгой согласованности.

Однако по мере роста нагрузки и усложнения требований к обработке, у этого подхода проявляются существенные ограничения, которые делают его малопригодным для высоконагруженных и отказоустойчивых систем [9].

1.2.2. Специализированные брокеры сообщений

Как было показано в предыдущем разделе, использование реляционных баз данных в качестве очереди сообщений сопряжено с существенными ограничениями по производительности, масштабируемости и гибкости маршрутизации. Для решения этих проблем в современных высоконагруженных системах применяются специализированные программные продукты — брокеры сообщений. Они представляют собой промежуточное ПО, обеспечивающее надёжную асинхронную передачу данных между независимыми компонентами, выступая в роли центрального хаба коммуникации.

Далее мы рассмотрим архитектуры таких брокеров как RabbitMQ и Apache Kafka.

1.2.2.1. RabbitMQ

Архитектура RabbitMQ базируется на взаимодействии двух основных компонентов: обменников и очередей сообщений. Обменники функционируют как интеллектуальные маршрутизаторы, которые на основе

заданных правил направляют поступающие сообщения в соответствующие очереди. Данный механизм обеспечивает архитектурную гибкость: поддержка различных типов обменников позволяет адаптировать систему под широкий спектр сценариев взаимодействия между сервисами. Отличительной особенностью RabbitMQ является его способность работать с распространенными протоколами обмена сообщениями, такими как AMQP, MQTT и STOMP. Это позиционирует брокер как универсальное средство для интеграции гетерогенных систем, требующих реализации сложной логики маршрутизации, а также для приложений, в которых применяются разнообразные паттерны обмена сообщениями [10].

1.2.2.2. Apache Kafka

Архитектура Apache Kafka базируется на концепции распределенного журнала сообщений. Данные в Kafka организованы в тематические разделы — топики, которые, в свою очередь, делятся на партиции для обеспечения параллелизма обработки и распределения нагрузки. Все сообщения сохраняются на диске, а их репликация автоматически регулируется между узлами кластера, что обеспечивает высокую степень отказоустойчивости системы.

Kafka демонстрирует наибольшую эффективность в сценариях, требующих обработки потоков данных большой интенсивности: система способна обрабатывать миллионы сообщений в секунду. Ключевым преимуществом данной архитектуры является гарантированное сохранение порядка сообщений в пределах одной партиции, что является критически важным требованием для множества бизнес-процессов. Кроме того, платформа поддерживает долговременное хранение информации. Указанные характеристики обусловливают применение Kafka преимущественно в

высоконагруженных системах с большими объемами данных, а также в приложениях, ориентированных на потоковую обработку информации в режиме реального времени [10].

1.2.3. Сравнительный анализ существующих решений

Каждая из представленных технологий имеет свои особенности и сильные стороны, но как же выбрать между этими тремя решениями на практике? Давайте проведем детальное сравнение по основным параметрам, которые критически важны при выборе брокера сообщений для определенных проектов. В таблице ниже рассматриваются критически важные параметры и сведения для каждого из брокеров.

Результаты сравнительного анализа представлены ниже (см. Таблица 1).

Критерий	Реляционная база данных как очередь	RabbitMQ	Apache Kafka
Потребление ресурсов сервера	Низкое. Используется уже существующий сервер БД	Среднее [10]. Требуется отдельный процесс/контейнер для брокера. Написан на языке, который использует Erlang VM, что увеличивает потребление памяти	Среднее [10]. Требуется отдельный процесс/контейнер. Написан на языке, который использует JVM, что увеличивает потребление памяти

Необходимость отдельного сервера	Нет. Используется та же СУБД, что и для хранения основных данных	Да. Нужен отдельный брокер	Да. Нужен отдельный кластер Kafka [10]
Время изучения	Низкое. Используется знакомый любому разработчику SQL	Среднее. Требуется устройство RabbitMQ	Среднее. Требуется изучить устройство Kafka
Производительность	Низкая/Средняя. Подходит до ~1,000 задач/с [9]	Средняя. Подходит до ~45,000 задач/с [10]	Очень высокая. Подходит до ~1,000,000 задач/с [10]
Гибкость маршрутизации	Отсутствует. Все задачи кладутся в таблицы, а выборка происходит по статусу. Любая маршрутизация будет самодельной реализацией разработчика [9]	Очень гибкая [10]	Базовая [10]
Устойчивость к сбоям	Зависит от настроек СУБД. При сбое СУБД	Высокая [10]	Высокая [10]

	очередь тоже падает		
Наличие готовых библиотек и инструментов	Есть драйверы для всех языков программирования. В некоторых СУБД существуют различные способы мониторинга очереди (LISTEN/NOTIFY в PostgreSQL) [9]	Доступна на всех основных языках, имеет богатую экосистему [10]	Доступна на всех основных языках, имеет богатую экосистему [10]

Проведённый обзор показывает, что специализированные брокеры сообщений, такие как RabbitMQ и Apache Kafka, предоставляют развитые механизмы маршрутизации, высокую производительность и масштабируемость, необходимые для построения отказоустойчивых высоконагруженных систем. RabbitMQ благодаря гибкой системе обменников и подтверждений хорошо подходит для задач со сложной логикой доставки и требованиями гарантий обработки. Kafka, в свою очередь, обеспечивает колоссальную пропускную способность и долговременное хранение событий, что делает её стандартом для потоковой обработки данных и сбора логов в крупных интернет-сервисах.

Однако применение этих брокеров сопряжено с рядом требований: они нуждаются в развёртывании отдельных сервисов, потребляют дополнительные вычислительные ресурсы (оперативную память, дисковое

пространство, процессорное время), а также требуют от разработчика изучения новых парадигм и инструментов. В условиях, когда проектируемая система рассчитана на невысокую нагрузку (порядка десятков или сотен задач в сутки), а инфраструктура ограничена виртуальным сервером с минимальными характеристиками, использование тяжёлых брокеров может оказаться избыточным и неоправданно ресурсоёмким.

В таких сценариях допустимым и прагматичным решением становится применение уже имеющейся реляционной базы данных (например, PostgreSQL [9]) для организации очереди задач. Хотя такой подход уступает профессиональным брокерам в производительности при высоких нагрузках и не поддерживает сложную маршрутизацию, он обладает рядом преимуществ в контексте малых проектов:

1. Отсутствие дополнительной инфраструктуры. Очередь работает в той же СУБД, что и основное приложение, не требуя установки и администрирования новых компонентов.
2. Минимальное потребление ресурсов. Не запускаются лишние процессы, нагрузка на сервер практически не возрастает по сравнению с обычной работой базы данных.
3. Простота реализации. Разработчик использует знакомый SQL и существующие библиотеки доступа к данным, не тратя время на изучение новых API.
4. ACID-гарантии. Возможность атомарно изменять статус задачи и связанные данные в единой транзакции, что критично для некоторых бизнес-процессов.

1.2.4. Итоги анализа

Таким образом, выбор между реляционной БД и профессиональным брокером должен определяться контекстом: ожидаемой нагрузкой, доступными ресурсами, сложностью предметной области и квалификацией команды. Для масштабных систем с высокими требованиями к производительности и отказоустойчивости предпочтительны RabbitMQ или Kafka. Для небольших проектов, работающих в условиях ограниченных ресурсов и не предъявляющих жёстких требований к пропускной способности, использование реляционной базы данных в качестве очереди является вполне оправданным решением, позволяющим сосредоточиться на реализации бизнес-логики без излишних инфраструктурных затрат.

1.3. Принципы работы S3-совместимых объектных хранилищ (MinIO, Garage, SeaweedFS, Ceph)

1.4. Сравнительный анализ существующих решений и обоснование выбора технологий

Глава 2. Проектирование архитектуры веб-приложения

2.1. Схема взаимодействия компонентов сервиса

2.2. Проектирование базы данных

2.3. Определение функциональных требований и REST API

2.4. Организация асинхронной обработки задач: двухуровневая очередь и планировщик Spring

2.5. Пул воркеров с ограниченным числом потоков

Глава 3. Реализация ключевых компонентов

3.1. Серверная часть приложения: Java 25, Spring Boot 4, Spring Data JPA

3.2. Асинхронная обработка: ThreadPoolTaskExecutor, планировщик задач

3.3. Интеграция с S3-хранилищем через AWS SDK v2

3.4. Конвертация медиафайлов с использованием FFMpeg через JavaCV

3.5. Клиентская часть приложения: TypeScript, React, паттерн «конечный автомат», взаимодействие с API

3.6. Контейнеризация и развёртывание: Docker, docker-compose, reverse proxy Caddy, HTTPS

3.7. Тестирование основных сценариев работы и оценка производительности

Заключение

Список использованной литературы

1. Глоссарий // The Reactive Manifesto = Манифест реактивных систем : [сайт]. – 2025. – URL:https://reactivemanifesto.org/ru/glossary (дата обращения: 09.03.2026). – Текст: электронный.

2. Message Queue // RisingWave : [сайт]. – 2025. – URL:https://www.risingwave.com/glossary/message-queue (дата обращения: 13.03.2026).

3. Event-Driven Architecture (EDA) // RisingWave : [сайт]. – 2025. – URL:https://risingwave.com/glossary/event-driven-architecture-(eda) (дата обращения: 13.03.2026).

4. Urma, R.-G. Modern Java in Action / R.-G. Urma, M. Fusco, A. Mycroft. – Shelter Island: Manning Publications, 2019. – 592 с. – ISBN 9781617293566. – Текст: непосредственный.

5. Kafka: The Definitive Guide: Real-Time Data and Stream Processing at Scale / G. Shapira, T. Palino, R. Sivaram, K. Petty. – Sebastopol: O'Reilly Media, 2021. – 488 с. – ISBN 9781492043089. – Текст: непосредственный.

6. Cleary, S. Concurrency in C# Cookbook: Asynchronous, Parallel, and Multithreaded Programming / S. Cleary. – 2nd ed. – Sebastopol: O'Reilly Media, 2019. – 254 с. – ISBN 9781492054504. – Текст: непосредственный.

7. RabbitMQ Documentation / VMware Inc. – 2026. – Текст: электронный. – URL:https://www.rabbitmq.com/docs (дата обращения: 15.03.2026).

8. Gorelick, M. High Performance Python: Practical Performant Programming for Humans / M. Gorelick, I. Ozsvald. – Sebastopol: O'Reilly Media, 2014. – 370 с. – ISBN 978-1-449-36159-4. – Текст: электронный. – URL:https://pepa.holla.cz/wp-content/uploads/2016/08/High-Performance-Python.pdf (дата обращения: 15.03.2026).

9. Postgres CDC Without Debezium: Build SQS/Kafka in SQL // Stacksync : [блог компании]. – 2026. – 4 янв. – Текст: электронный. –

URL:https://www.stacksync.com/blog/postgres-cdc-without-debezium-build-sqskafka-sql (дата обращения: 15.03.2026).
10. Оптимизация взаимодействия микросервисов с использованием асинхронных вызовов и брокеров сообщений (Kafka, RabbitMQ) / А. Э. Баженов, А. В. Райков, М. В. Нехаев, Н. В. Ореховский // Программные системы и вычислительные методы. – 2025. – № 4. – С. 77-93. – DOI 10.7256/2454-0714.2025.4.77228. – EDN URNSWS. – Текст: электронный. – URL:https://ru.aurora-journals.com/library_read_article.php?id=77228 (дата обращения: 16.03.2026).

Приложение